中山大學
SUN YAT-SEN UNIVERSITY



C++多态

中山大学计算机学院

主讲人：？？？

？？？@mail.sysu.edu.cn

目录

CONTENTS

01 虚函数及其意义

02 多态的概念

03 多态-函数重载

04 多态-动态类型

05 抽象类

06 虚析构

07 RTTI运行时类型识别机制

08 final 关键字（C++11）

09 多态的实现原理（选讲）

问题引入：如何处理抽象概念

张三的**疑惑：这程序不符合日常认知逻辑！**

```
//… class definitions of Animal,Cat,Dog etc.
int main() {
    Animal &&ani=Animal(), &&dog=Dog();
    ani.eat();  //问题
    dog.eat();
}
```

问题1：ani指代的是一(个，只，条…？)(动物…猫，鱼，狗… 四不像？)

问题2：Animal 都不知道自己是什么，真的知道喜欢吃什么吗?

问题引入：如何处理抽象概念

一种解决方案：

Abstract_type_origin.cpp

```
class Animal {
    public:
        virtual void eat() {//②别 call 我
            throw std::exception();
        };
    protected:
        Animal() = default; //①阻止用户构造对象
};
```

要点1：保护构造函数，使得client无法构造抽象对象。只能用 animal 的指针或引用，指向具体的事物

要点2：基类虚函数抛出异常，让派生类必须覆盖基类的虚函数

纯虚函数 与 抽象类

C++解决方案：

Abstract_type_pure.cpp

```
class Animal {
    public:
        virtual void eat() = 0;
};
```

纯虚函数：用“=0”作为虚函数申明的后缀，表示该函数是纯虚函数

- 纯虚函数可以（不建议）给定义。（若纯虚函数是析构函数则必须提供）

抽象类（Abstract Class）：定义或继承了至少一个纯虚函数的类。

- 抽象类不能被实例化
- 抽象类的子类是抽象类，除非**所有纯**虚函数都被覆写（Override）为非纯虚函数

抽象类：官方示例

```
struct Abstract {
    virtual void f() = 0; // 纯虚
}; // "Abstract" 为抽象

struct Concrete : Abstract {
    void f() override {} // 非纯虚
    virtual void g();    // 非纯虚
}; // "Concrete" 为非抽象

struct Abstract2 : Concrete {
    void g() override = 0; // 纯虚覆盖函数
}; // "Abstract2" 为抽象
```

```
int main()
{
    // Abstract a; // 错误：抽象类
    Concrete b; // OK
    Abstract& a = b; // OK：到抽象基类的引用
    a.f(); // 虚派发到 Concrete::f()
    // Abstract2 a2; // 错误：抽象类（g() 的最终覆盖函数为纯虚）
}
```

Cppreference：抽象类

抽象类语法与语义

- **能仅能**作为**基类指针或引用**，因为不可能存在抽象对象
- 不能申明抽象类的对象。例如：Animal a
- 不能被显式转为抽象类对象。例如：（Animal）dog
- 不能作为函数参数类型或者返回的值。例如：func(Animal)
- 能申明为指针或引用，指代自己派生类对象

为多态而生!

案例研究：抽象类与接口（1）

Abstract_type_interface.cpp

```
class Shape {
    public:
        virtual void draw() = 0;
        //virtual ~Shape()  {};
};

class Shape2D:public virtual Shape {
    public:
        static int count;
        virtual float getArea() = 0;
        virtual float getPerimeter() = 0;
};

int Shape2D::count = 0;
```

C++**没有**接口的概念和定义。
Java接口是由一组函数申明和静态成员构成的特殊类。

C++可以模仿Java的定义描述接口产生类似的效果
（1）成员函数都是纯虚类
（2）可以包含静态成员，不包含任何自己的数据
（3）可以有虚析构函数，但不需要构造函数
（4）被虚继承保证唯一，多继承也不会命名冲突

接口的**优势**：
- 被继承或被多重继承的派生类必须覆盖实现它的成员
- 它自己没有数据和业务逻辑，可以无歧义地被多重继承
- 即使没有任何成员，也可作为任意类型对象的特征。例如：SYSUer 抽象类可作为与中大有时空交集的人的共有特征，如学生，老师，毕业生，进修生 …

案例研究：抽象类与接口（2）

Abstract_type_interface.cpp

```
class Circle:public virtual Shape2D {
        public:
                Circle(float r=1):r(r) {count++;};
                void draw() {
                        cout << "Circle, r = " << r << endl;
                }
                float getArea() {
                        return PI * r * r;
                }
                float getPerimeter(){
                        return 2 * PI * r;
                }
                ~Circle() {
                        count--;
                        cout << "Circle desctructed" << endl;
                }
        private:
                float r;
};
```

这里，Circle 是具有 Shape2D 特征的事物（继承接口具体类一般用**术语XX接口实现**），因此必须实现所有的虚函数，draw，getArea 和 getPerimeter。否则**实例化时**提示**编译错误**。

虚函数没有定义是**链接错误**。

案例研究：抽象类与接口（3）

Abstract_type_interface.cpp

```
int main() {
	Shape2D *c1 = new Circle();

	//EX: 请取消下面的注释，编译。根据编译提示，消除编译错误。
	// 再运行，请问结果对吗？ 为什么？
//	Shape2D *r1 = new Rectangle(2,3);
//	cout << "the rect's area is " << r1->getArea() << endl;
//	delete dynamic_cast<Rectangle*>(c1);

	cout << "the number of Shape2Ds is " << Shape2D::count << endl;
	delete dynamic_cast<Circle*>(c1);
}
```

案例研究：练习（判断客户端代码对错）

Abstract_type_interface.cpp

```
Shape fun1(int);  //①

void fun2(Shape2D a);  //②

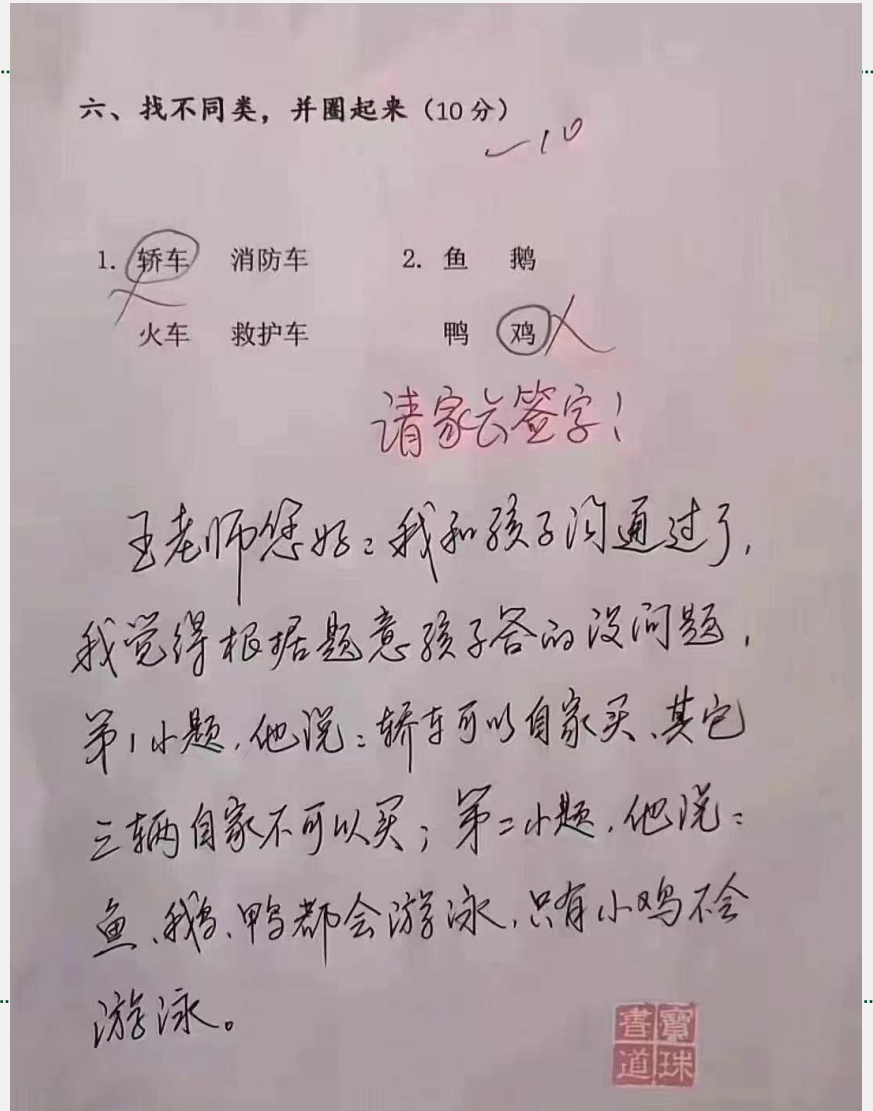
Shape & fun3(Shape2D &a);  //③

int main()
{
    Shape2D x; //④

    Shape *p; //⑤

    return 0;
}
```

世界的复杂性-笑话与多重继承与接口



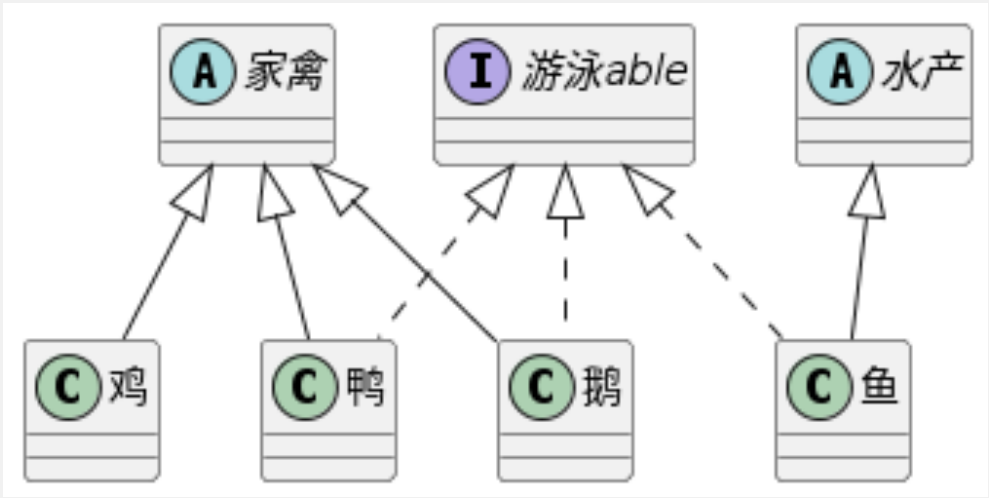
六、找不同类，并圈起来（10分）

1. 轿车　消防车　　2. 鱼　鹅

火车　救护车　　鸭　鸡

请家长签字！

王老师您好：我和孩子沟通过了，我觉得根据题意孩子答的没问题，第1小题，他说：轿车可以自家买，其它三辆自家不可以买；第二小题，他说：鱼、鹅、鸭都会游泳，只有小鸡不会游泳。



目录

CONTENTS

01 虚函数及其意义

02 多态的概念

03 多态-函数重载

04 多态-动态类型

05 抽象类

06 虚析构

07 RTTI运行时类型识别机制

08 final 关键字（C++11）

09 多态的实现原理

虚析构函数

你可能注意到，程序回避了 **new 包含虚函数的类** 这样的语句，因为

- 在抽象类与接口案例中，多态类型指针必须动态转换为对象实际类型指针才能正确执行对象析构过程。
- EX1：修改 Abstract_type_interface.cpp，取消 dynamic_cast 观察对象是否被正确析构

C++提供了**虚析构函数**，解决这个问题

- 虽然析构函数是不继承的，但若基类声明其析构函数为 virtual，则派生的析构函数始终覆盖它。这使得可以通过指向基类的指针 delete 动态分配的多态类型对象
- EX2：在包含虚函数的基类添加虚析构函数，编译执行，观察多态指针析构过程
- 任何**包含虚函数的基类**的**析构函数必须为公开且虚，或受保护且非虚。** **否则很容易导致内存泄漏**

RTTI 概念

RTTI (Run Time Type Identification) 即通过**运行时类型识别**，程序能够使用**基类的指针或引用**来**检查**着这些指针或引用所指的**对象的实际派生类型**。

- C 是一种**静态类型**语言。其数据类型是在编译期就确定的，不能在运行时更改。
- 面向对象程序设计中**多态性**要求，C++中的**指针或引用**本身的类型，可能与它实际指代(指向或引用)的类型并不一致，需要在运行时将一个多态指针转换为其实际指向**对象的类型**。

RTTI 提供了两个非常有用的**操作符**：**typeid** 和 **dynamic_cast**。

- typeid 操作符，返回指针和引用所指的实际类型（type_info 对象）；
- dynamic_cast 操作符，将基类类型的指针或引用**安全地转换**为其派生类类型的**指针或引用**。

关键词：typeid 运算符

查询类型的信息。用于必须知晓多态对象的动态类型的场合以及静态类型鉴别。

- 在使用 typeid 前，必须包含头文件 **<typeinfo>**
- typeid 返回 std::type_info 对象，它常用的有 ==、!= 运算符 和 name() 成员

语法1：**typeid (类型)**

- 指代一个表示 *类型* 类型的 std::type_info 对象。若 *类型* 为引用类型，则结果所指代的 std::type_info 对象表示被引用的类型。。

语法2：**typeid (表达式)**

a) 若 *表达式* 为标识某个**多态类型**（即声明或继承至少一个虚函数的类）对象的**泛左值表达式**，则 typeid 表达式对该表达式**求值**，然后指代表示该表达式**动态类型的 std::type_info** 对象。

b) 若 *表达式* **不是多态类型**的泛左值表达式，则 typeid **不**对该表达式**求值**，而是由编译静态推导表达式静态类型的 std::type_info 对象

Cppreference： typeid 运算符

关键词：dynamic_cast 类型转换运算符

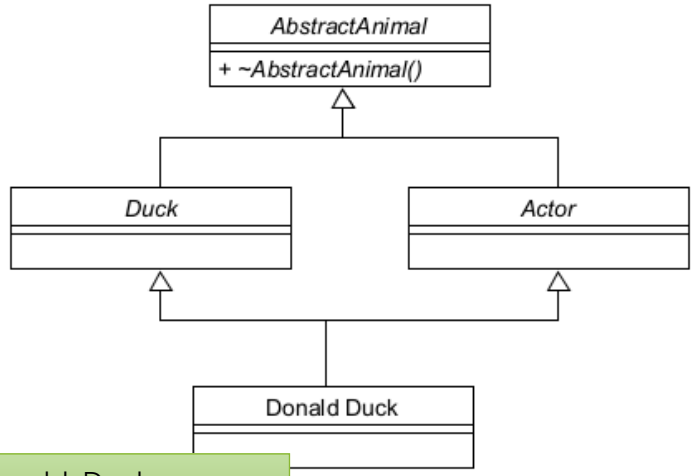
沿继承层级**向上**、**向下**及**侧向**，安全地转换到其他类的指针和引用。。

语法：**dynamic_cast < 新类型 > (表达式)**

若转型成功，则 dynamic_cast 返回 新类型 类型的值。

若转型失败，

- 且 新类型 是指针类型，则它返回该类型的**空指针**。
- 且 新类型 是引用类型，则它**抛出**与类型 **std::bad_cast** 的处理块匹配的**异常**。



RTTI_Donald_Duck.cpp

```
int main() {
    //向上
    Animal *a = new Donald();
    //向下
    Duck *d = dynamic_cast<Duck*>(a);
    //侧向
    Actor *c = dynamic_cast<Actor*>(d);
    delete c;
}
```

Cppreference： dynamic_cast 转换

关键字：final 说明符 (C++11 起)

指定某个虚函数不能在子类中被覆盖，或者某个类不能被子类继承。官方例子

```
struct Base {
    virtual void foo();
};
struct A : Base {
    void foo() final; // Base::foo 被覆盖而 A::foo 是最终覆盖函数
    void bar() final; // 错误： bar 不能为 final 因为它非虚
};

struct B final : A  { // struct B 为 final
    void foo() override; // 错误：foo 不能被覆盖，因为它在 A 中是 final
};

struct C : B { // 错误：B 为 final
};
```

Cppreference：final 说明符

目录

CONTENTS

01 虚函数及其意义

02 多态的概念

03 多态-函数重载

04 多态-动态类型

05 抽象类

06 虚析构

07 RTTI运行时类型识别机制

08 final 关键字（C++11）

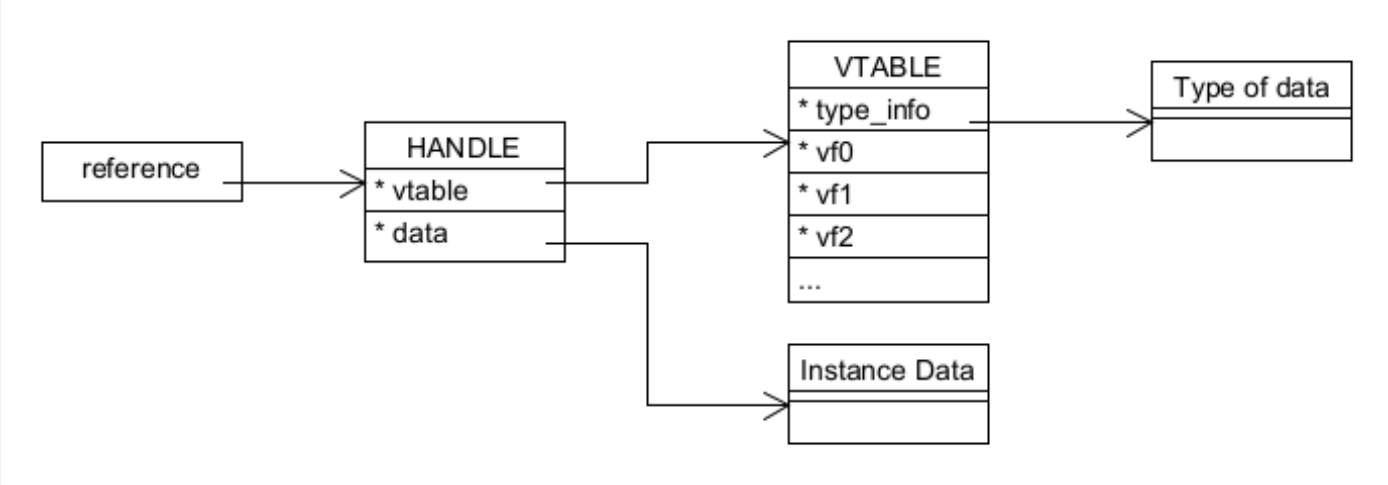
09 多态的实现原理（选讲）

多态实现-基本原理

Java多态性实现机制：

In Sun's current implementation of the Java Virtual Machine, **a reference** to a class instance is **a pointer to a handle that is itself a pair of pointers**: one to a table containing the **methods of the object** and a pointer to the Class object that represents the **type of the object**, and **the other to** the memory allocated from the Java heap for the **object data**. （jvm规范中关于对象内存布局的说明）

SUN目前的JVM实现机制，类实例的**引用**是一个句柄（handle）的指针，这个句柄是**一对指针**：**一个指针**指向一张表格，这个表格包括对象的**方法表**和一个指针指向对象的**类型表示对象**；**另一个指针**指向堆中分配出来的**对象数据**

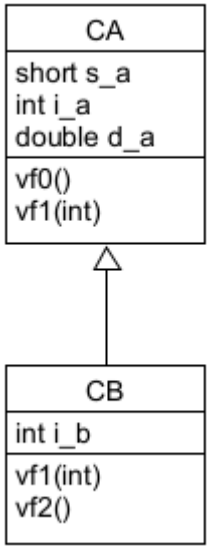


多态实现-C++实现（1）

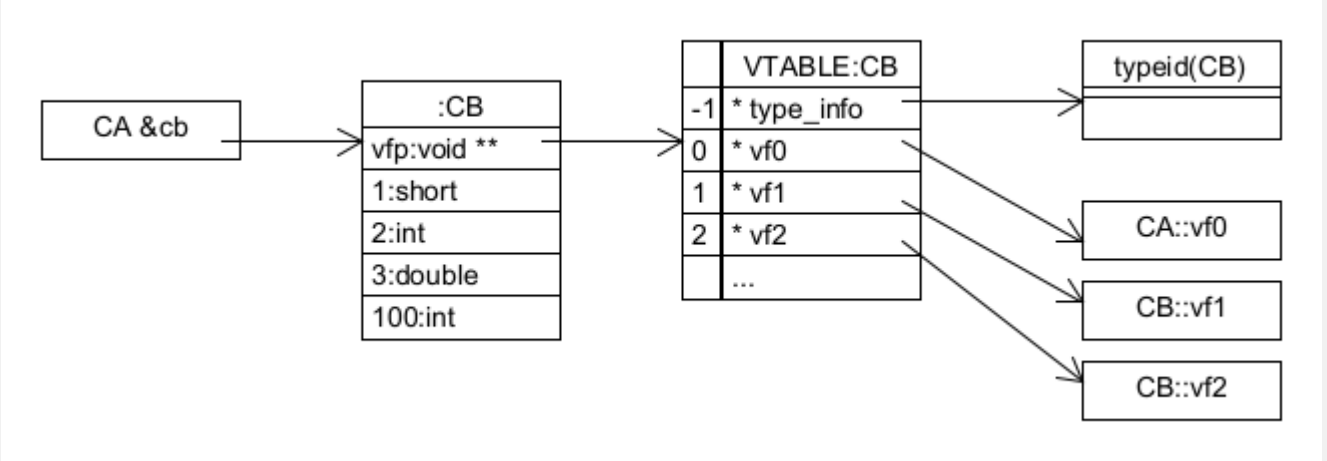
Virtual_func_impl.cpp

C++从来没有公布其多态性实现机制，以下均根据网络资源整理：

简单地只考虑**单继承**情况，**多态类型**基类 CA 和 派生类 CB，其所有**成员函数为虚函数。**



类的定义



语句 CA &&cb = CB（） 执行后相关对象的内存布局示意图

多态实现-C++实现（2）

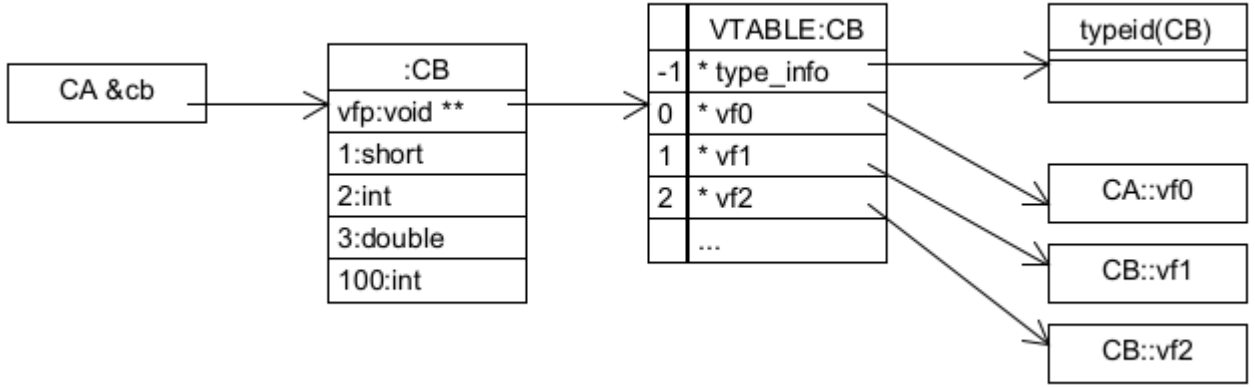
Virtual_func_impl.cpp

C++多态性实现机制要点：

- 三个对象：
  - 数据对象，包含指向虚表的指针
  - 虚函数表（每个类型一个张）
    - 其索引是虚函数的申明顺序
    - -1 位置是实际对象类型描述对象
  - 类型描述类（type_info）派生类的实例
- 三个指针
  - *ptr 多态类型指针
  - *vfp 指向实际类型虚函数表的指针
  - *type_info 指向类型描述对象的指针

（以下严重超纲）

- 多态类型使用 sizeof 操作，结果包括vfp指针空间
- 无非静态数据成员的类 sizeof(T) = 1; 被继承后，被优化不占空间



语句 CA &&cb = CB（） 执行后相关对象的内存布局示意图

多态实现-C++实现（3）

Virtual_func_impl.cpp

程序阅读提示（注意：不同编译器可能实现不同，可能得不到期望结果）：

- 编译在运行前准备好的工作（以下自动加载到内存）：
  - 将所有多态类型和 typeid 使用的静态类型，生成一张 type_info* 常量表
  - 为每种多态类型生成对应虚函数表
- 多态对象的初始化大致过程
  1. 分配对象空间，可能包括引用、虚函数指针需要的空间
  2. 基类初始化，如果基类是多态类，迭代执行2-3
  3. 执行成员初始化，设置当前类型 vfp， 执行构造函数题。（见上周理论题第四题）
- 程序中难点
  - 将一个 非**标准布局类型**对象 转为一个 **POD**（plain old data）即 **C 语言结构体**
  - **函数指针，任意参数函数，void* 数组，各种不安全的类型转换**

多态实战案例 – 菜单

见附件程序和说明PPT

中山大學
SUN YAT-SEN UNIVERSITY



谢谢

中山大学计算机学院

中山大学MOOC课程组